If you want to exclude tags, you prefix the tag name with a tilde:

```
$ rspec --tag ~focus
```

RSpec's default spec_helper file contains a commented-out line that applies the :focus filter automatically if any specs in the system are marked as :focus, which is convenient if you like using focus.

There isn't much that's special about the focus tag; you can use metadata to add any tag you want in RSpec with the --tag command line option. For example, here's a fun thing you can do:

environment/01/spec/support/profiling.rb

```
class UnclaimedSlowTestException < RuntimeError
  THRESHOLD = 1

  def initialize(example)
    @example = example
  end

  def message
    "This spec at #{@example.metadata[:location]}
     is slower than #{THRESHOLD} seconds.
     Either make it faster or mark it as :slow in metadata"
  end
end

RSpec.configure do |config|
  config.append_after(:each) do |ce|
    runtime = ce.clock.now - ce.metadata[:execution_result].started_at
    if runtime > UnclaimedSlowTestException::THRESHOLD && !ce.metadata[:slow]
      raise UnclaimedSlowTestException.new(ce)
    end
  end
end
```

The exact internals of this are beside the point. What it does is this: every test that's slower than a threshold value (here it's 1 second) fails unless it's explicitly tagged as :slow in its metadata. The nice thing about this is you can then run rspec --tag ~slow and only run your fast tests when you want.

Prescription 44 Use RSpec metadata like :focus or :slow to be able to run arbitrary groups at once.

If the config.example_status_persistence_file_path = "spec/examples.txt" option in the spec_helper.rb is uncommented, then every time you run your specs, you get a file that includes a notation for every spec in the system, whether it passed or failed on its last run, and how long it took.